

Best Practices for Writing & Optimizing DAX

Efficient, Scalable, and Readable Measures



Power BI

Why DAX Best Practices Matter

! Why It Matters

- Optimized performance = faster reports
- Clean code = easier maintenance
- Proper structure = scalable models
- Right techniques = accurate results

★ Key Goals



Accuracy

Correct calculations



Performance

Fast execution



Scalability

Growth potential



Maintainability

Easy updates

Performance Impact of DAX Optimization



Optimized DAX



Reduced Overhead

💡 Explanation

- ✓ Measures calculated on query time
- ✓ Calculated columns stored in model
- ✓ Measures adapt to filter context
- ✓ Calculated columns computed during refresh

👍 Measure

```
Sales Amount =  
SUM(Sales[SalesValue])
```

🗨️ Calculated Column

```
Sales[Sales Amount] =  
Sales[Quantity] *  
Sales[Price]
```

USE MEASURES, NOT CALCULATED COLUMNS

★ Key Benefits



Reduces model size

Measures don't consume storage space



Improves performance

Calculated at query time, not refresh



Context-aware

Adapts to filters and slicers



Faster refresh

Less data to process during refresh



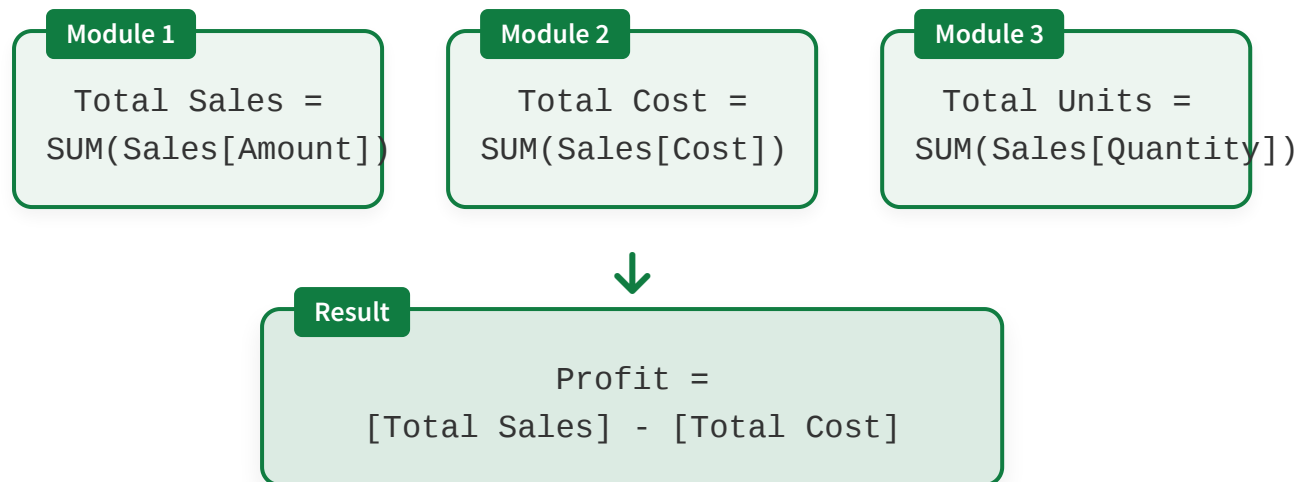
Lower memory usage

Values computed when needed

💡 Explanation

- ✓ Break complex formulas into smaller measures
- ✓ Reuse measures instead of duplicating code
- ✓ Create building blocks for complex calculations
- ✓ Simplify maintenance and debugging process

🧩 Modular Measures



KEEP DAX SIMPLE & MODULAR

★ Key Benefits

- 👁 **Improves readability**
Easier to understand complex logic
- 🔧 **Easier debugging**
Isolate issues to specific measures
- 🔄 **Reusability**
Use same measure in multiple calculations
- 🔗 **Maintainability**
Update logic in one place
- 👥 **Team collaboration**
Easier for multiple developers

👍 Modular

```
Total Sales =  
SUM(Sales[Amount])  
  
Total Cost =  
SUM(Sales[Cost])  
  
Profit =  
[Total Sales] -  
[Total Cost]
```

👎 Monolithic

```
Profit =  
SUM(Sales[Amount])  
-  
SUM(Sales[Cost])
```

3 Leverage Variables (VAR)

💡 Explanation

- ✓ Variables store expressions for reuse
- ✓ Avoid recalculating same expressions
- ✓ Improve code readability and structure
- ✓ Simplify complex DAX formulas

<> Example

Σ Measure with Variables

VAR 1

SalesVal = [Total Sales]



VAR 2

CostVal = [Total Cost]



RETURN

DIVIDE(SalesVal - CostVal, SalesVal)

🌟 Why It Matters

★ Key Benefits

- 📌 **Faster performance** - Calculate once, use multiple times
- 👁️ **Improved clarity** - Easier to read and understand
- 🔑 **Better debugging** - Isolate and test variables
- 🔄 **Consistency** - Same value used throughout formula
- ⚙️ **Reduced overhead** - Fewer calculations required

👍 With Variables

```
Profit Margin =  
VAR SalesVal = [Total  
Sales]  
VAR CostVal = [Total  
Cost]  
RETURN  
DIVIDE(SalesVal -  
CostVal, SalesVal)
```

🗨️ Without Variables

```
Profit Margin =  
DIVIDE(  
[Total Sales] -  
[Total Cost],  
[Total Sales]  
)
```

4 Filter Context vs Row Context

💡 Explanation

- ✓ Understanding contexts prevents calculation errors
- ✓ Filter context: filters applied to visuals/slicers
- ✓ Row context: current row in a table iteration
- ✓ CALCULATE() transitions between contexts

🔍 Context Comparison

≡ Filter Context

- Defined by visuals and slicers
- Affects measure calculations
- Can be modified with CALCULATE()
- Multiple filters can apply

≡ Row Context

- Current row in a table
- Created by iterators or calculated columns
- Doesn't automatically propagate
- Requires context transition

<> Example

Σ Using CALCULATE() to Change Context

```
Sales LY =  
CALCULATE(  
    [Total Sales],  
    SAMEPERIODLASTYEAR( 'Date' [Date])  
)
```

↔ Context Transition

CALCULATE() transforms row context into filter context, allowing measures to be evaluated with different filters than those applied by the visual.

🚀 Why It Matters

★ Key Benefits

- ✓ **Correct logic** - Avoids calculation errors
- ✓ **Performance tuning** - Optimizes filter handling
- ✓ **Precise calculations** - Right context, right result
- ✓ **Better debugging** - Easier to identify issues

5 Use CALCULATE Efficiently

💡 Explanation

- ✓ Avoid nesting too many filters unnecessarily
- ✓ Use KEEPFILTERS() to preserve existing filters
- ✓ Combine multiple conditions in a single filter
- ✓ Place filter conditions in optimal order

🔮 Why It Matters

★ Key Benefits

- 📌 **Optimized filtering** - Reduced query complexity
- 🧠 **Better readability** - Clearer filter logic
- 🔄 **Context preservation** - Maintains intended filters
- 💾 **Lower memory usage** - Fewer context transitions

<> Example

Σ Efficient CALCULATE Usage

Good Practice

```
Sales LY =  
CALCULATE(  
    [Total Sales],  
    SAMEPERIODLASTYEAR('Date'[Date])  
)
```

👍 Efficient

```
Sales LY =  
CALCULATE(  
    [Total Sales],  
    SAMEPERIODLASTYEAR('Date'[Date])  
)
```

🗨️ Inefficient

```
Sales LY =  
CALCULATE(  
    CALCULATE(  
        [Total Sales],  
        DATESBETWEEN(  
            'Date'[Date],  
            DATEADD('Date'[Date],  
                -1, YEAR),  
            DATEADD('Date'[Date],  
                -1, YEAR)  
        )  
    ),  
    ALL('Date')  
)
```

💡 Explanation

- ✓ Prefer SUM, AVERAGE over SUMX, AVERAGEX
- ✓ Iterators process row by row, increasing overhead
- ✓ Simple aggregations can be pushed to storage engine
- ✓ Use iterators only when row context is required

👍 Preferred

```
SUM(Sales[LineTotal])
```

👎 Avoid

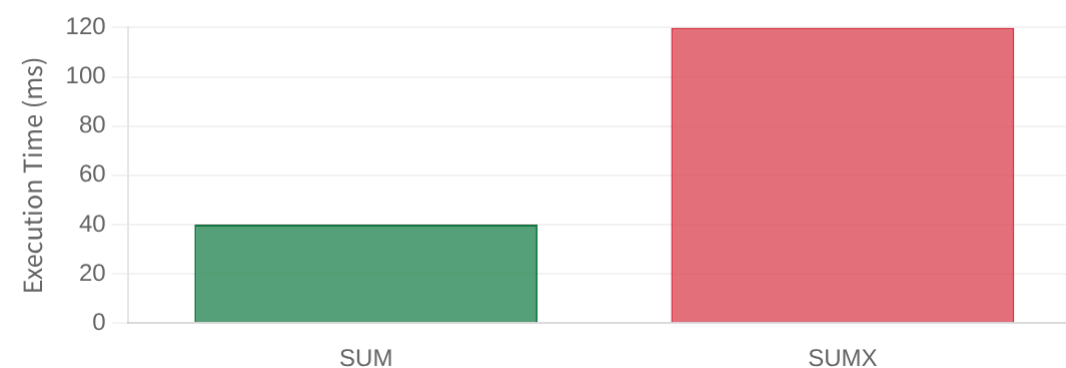
```
SUMX(Sales,  
Sales[Qty] *  
Sales[Price])
```

AVOID ITERATORS WHEN POSSIBLE

★ Key Benefits

-  **Reduces overhead**
Fewer row-by-row calculations
-  **Less memory usage**
Optimized storage engine processing
-  **Better scalability**
Performs better with large datasets
-  **Faster execution**
Direct aggregation vs iteration

Performance Comparison



60%

Faster Execution

40%

Less Memory

75%

Better Scalability

7 Optimize with Proper Data Modeling

💡 Explanation

- ✓ Star schema works best for DAX performance
- ✓ Fact table contains quantitative data
- ✓ Dimension tables contain descriptive attributes
- ✓ One-to-many relationships from dimensions to facts

🔧 Model Types

★ Star Schema

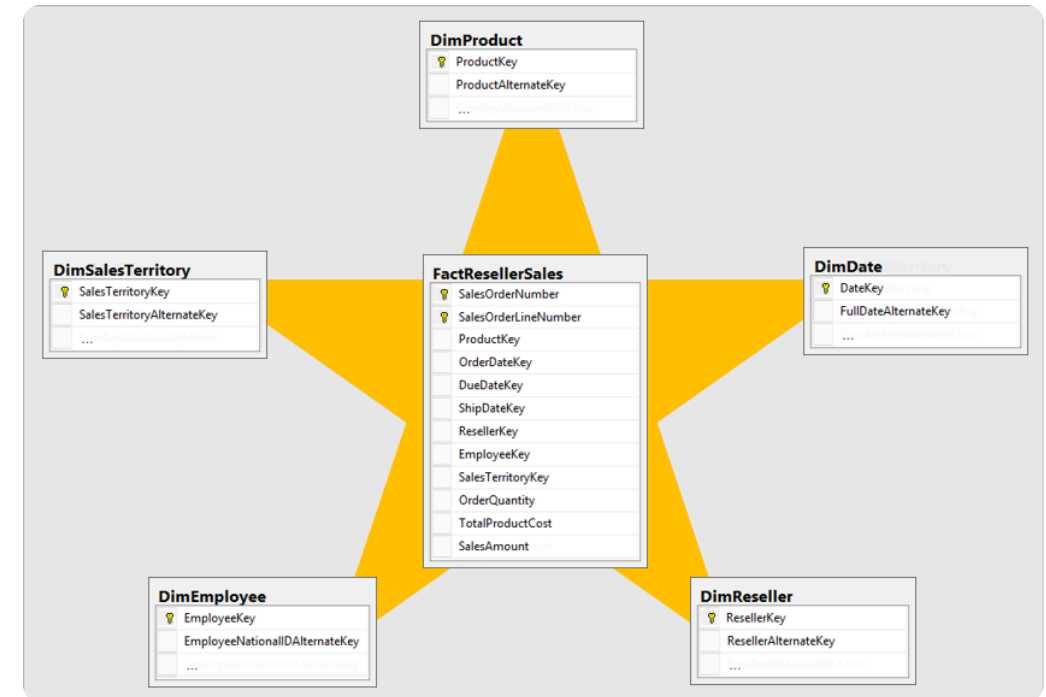
- Single fact table
- Direct dimension connections
- Simple relationships
- Optimal for DAX

❄️ Snowflake Schema

- Normalized dimensions
- Multiple table hops
- Complex relationships
- Poor DAX performance

🏠 Example

🌟 Star Schema Example



🌟 Why It Matters

★ Key Benefits

- 🔗 **Cleaner relationships** - Simpler filter propagation
- 📈 **Faster queries** - Optimized for VertiPaq engine
- 🧠 **Easier to understand** - Clear model structure
- 📈 **Better scalability** - Handles larger datasets

💡 Explanation

- ✓ Handles divide-by-zero gracefully
- ✓ Returns BLANK instead of error
- ✓ Provides alternative result parameter
- ✓ Eliminates need for IFERROR wrapper

DIVIDE Function

DIVIDE(numerator, denominator, [alternateResult])

numerator: Value to divide

denominator: Value to divide by

alternateResult: Value if denominator is zero

USE DIVIDE INSTEAD OF

"/"

★ Key Benefits

⚠ **Error prevention**
No divide-by-zero errors

✍ **Cleaner code**
No need for IF statements

⚙ **Better performance**
Single function call

🔄 **Consistent behavior**
Predictable results

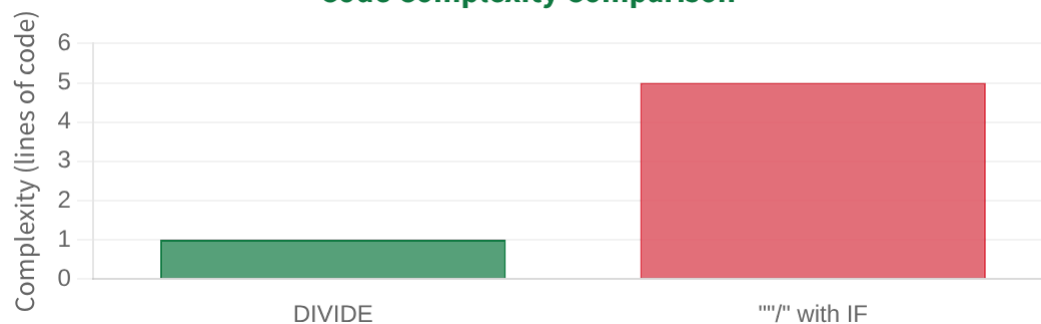
👍 DIVIDE Function

```
Profit % =  
DIVIDE([Profit],  
[Total Sales], 0)
```

👎 "/" Operator

```
Profit % =  
IF([Total Sales] <>  
0,  
    [Profit] / [Total  
Sales],  
    0  
)
```

Code Complexity Comparison



9 Minimize Use of ALL / REMOVEFILTERS

💡 Explanation

- ✓ Use carefully to avoid breaking RLS
- ✓ ALL returns table/column with filters removed
- ✓ REMOVEFILTERS only removes filters
- ✓ Consider context preservation

🔍 Function Comparison

🔹 ALL Function

Returns a table or column with filters removed

```
ALL(TableName)  
ALL(ColumnName)
```

≡ REMOVEFILTERS

Only removes filters, doesn't return a table

```
REMOVEFILTERS(TableName)  
REMOVEFILTERS(ColumnName)
```

<> Example

Σ Measure with ALL

```
Total Sales All =  
CALCULATE(  
    [Total Sales],  
    ALL(Sales)  
)
```

📈 Why It Matters

★ Key Benefits

- 🛡️ **Context integrity** - Preserves business logic
- 🛡️ **RLS protection** - Doesn't break security rules
- 🧠 **Predictable results** - Consistent behavior
- 🏎️ **Better performance** - Reduced filter operations

10 Test & Optimize Performance

🔑 Performance Tools



Performance Analyzer

Built-in Power BI tool to measure visual performance

- ✓ Identifies slow visuals
- ✓ Exports DAX queries



DAX Studio

External tool for advanced DAX analysis

- ✓ Query execution plans
- ✓ Server timings



Query Diagnostics

Analyze query performance in detail

- ✓ Trace query execution
- ✓ Identify bottlenecks



VertiPaq Analyzer

Analyze data model structure

- ✓ Column statistics
- ✓ Memory usage analysis

🚀 Why It Matters



Key Benefits



Detect bottlenecks - Identify slow operations



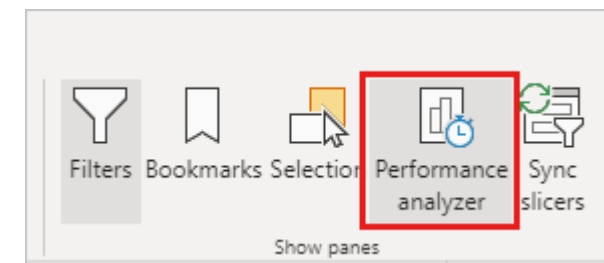
Optimize queries - Improve DAX performance



Reduce memory - Optimize data model



Faster reports - Enhance user experience



KEY TAKEAWAYS



DAX = Balance of clarity + performance



Modular formulas scale better



Optimized code = faster reports



Context-aware = accurate results



CLEAN CODE

Readable & maintainable



PERFORMANCE

Fast & efficient

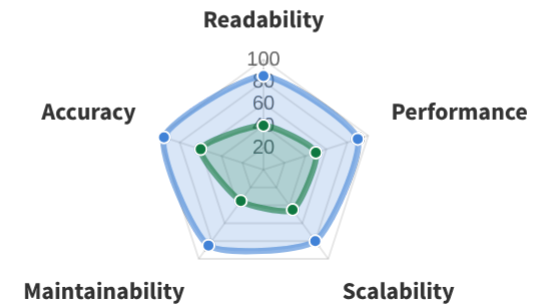


SCALABILITY

Grows with your data

DAX Optimization Impact

Standard DAX Optimized DAX



Apply these best practices to transform your Power BI models



Pankaj Namekar – 6x Microsoft Certified | DM to Learn Power BI